

NUS SAVNAC DOCUMENTATION

Team name **SchedWarden**

Project name **NUS Savnac**

1. Introduction

1.1. Motivation

The current learning ecosystem at the **National University of Singapore** relies heavily on platforms such as **Canvas** and external systems like **Coursemology**. However, course coordinators adopt different teaching workflows, resulting in inconsistent structures across modules. For instance, some modules rely on quizzes and assignments within Canvas, while others require students to access external platforms.

Furthermore, **Canvas** does not provide a unified view of all academic responsibilities. Important tasks such as tutorial preparation, lab work, and self-directed study are often not explicitly tracked within the system, requiring students to manage them manually. This becomes especially challenging for freshmen who are still adapting to university-level learning.

In addition, navigating course materials in **Canvas** – particularly within the *Files* section—can be inefficient due to deeply nested folder structures. Students frequently need to access these materials, making the process repetitive and time-consuming.

Lastly, **Canvas** lacks built-in tools for structured study planning and task scheduling. Students are often required to rely on external applications to organize their academic workload, leading to fragmented workflows.

These limitations increase cognitive load and reduce productivity, motivating the need for a centralized and student-centric system to streamline academic management.

1.2. Project Overview

NUS Savnac is a academic management website that is designed specifically for NUS students, helping them centralize their study resources, schedule, to-do list, etc.

The name *Savnac* is the reverse of the name *Canvas*, which is the platform that runs courses of NUS, allowing students to access courses' materials, viewing grades and announcements. **NUS Savnac** will play the role of an assistant, extending the limitations of Canvas, which will ease the heaviness of workload, making students' university life easier and more organizable.

NUS Savnac system has 4 main features:

- **Dashboard** - This feature contains multiple sub-features. Users can add new NUS courses, adding their own folders to the course that can link to external resources, especially the *Files* section of the course. The system also fetches data (schedule, workload, exam dates) of the course from NUSMods, and helps students view their weekly to-dos of the course.

- **Scheduler** - This page is a semester-based schedule planner that aligned with NUS Academic Calendar. It gathers the schedules of all added courses and displays them in one interface. Students can add their own events on certain dates, set up their study routines, as well as having a view of upcoming events.
- **Task** - This page collects the to-do lists of all courses and displays it in one interface. The page is designed in a way such that can help the students easily track their weekly progress, and motivate them to finish all the tasks.
- **Reminder System** - This small system will keep track of the upcoming deadlines and remind the users before the deadlines a specific amount of time depending on users' choice. The reminding message will be announced via the registered email.

The system is not limited to those 4 main features. It is also extended with other small features that can boost students productivity:

- **Pomodoro Timer** - This feature can help students queue a series of tasks they want to do and set the time limit they want to finish those tasks. This functionality can enhance focus and productivity.
- **AI Advising System** - Manually setting up the study routines in the scheduler as well as the to-dos for upcoming deadlines can be a bit of pain. Therefore, this advising system can suggest optimal study routines and to-do tasks so that students can follow.
- **Voice Command System** - This system contains a set of predefined voice commands which users can use to interact with the system instead of manual interaction.

1.3. Expectations

NUS Savnac is a mixture of many features that is designed for the better use of Canvas as well as enhancing students studies and university life. Therefore, this website is expected to fully:

- group students courses information, schedules, and to-dos appropriately so that it can ease the students' workload and avoid extra work.
- provide tools to enhance focus and productivity of users
- make sure students can have an overview of all their work and does not unintendedly miss any deadlines.

2. System Design

2.1. Overall Architecture

The overall architecture of the system is shown below.

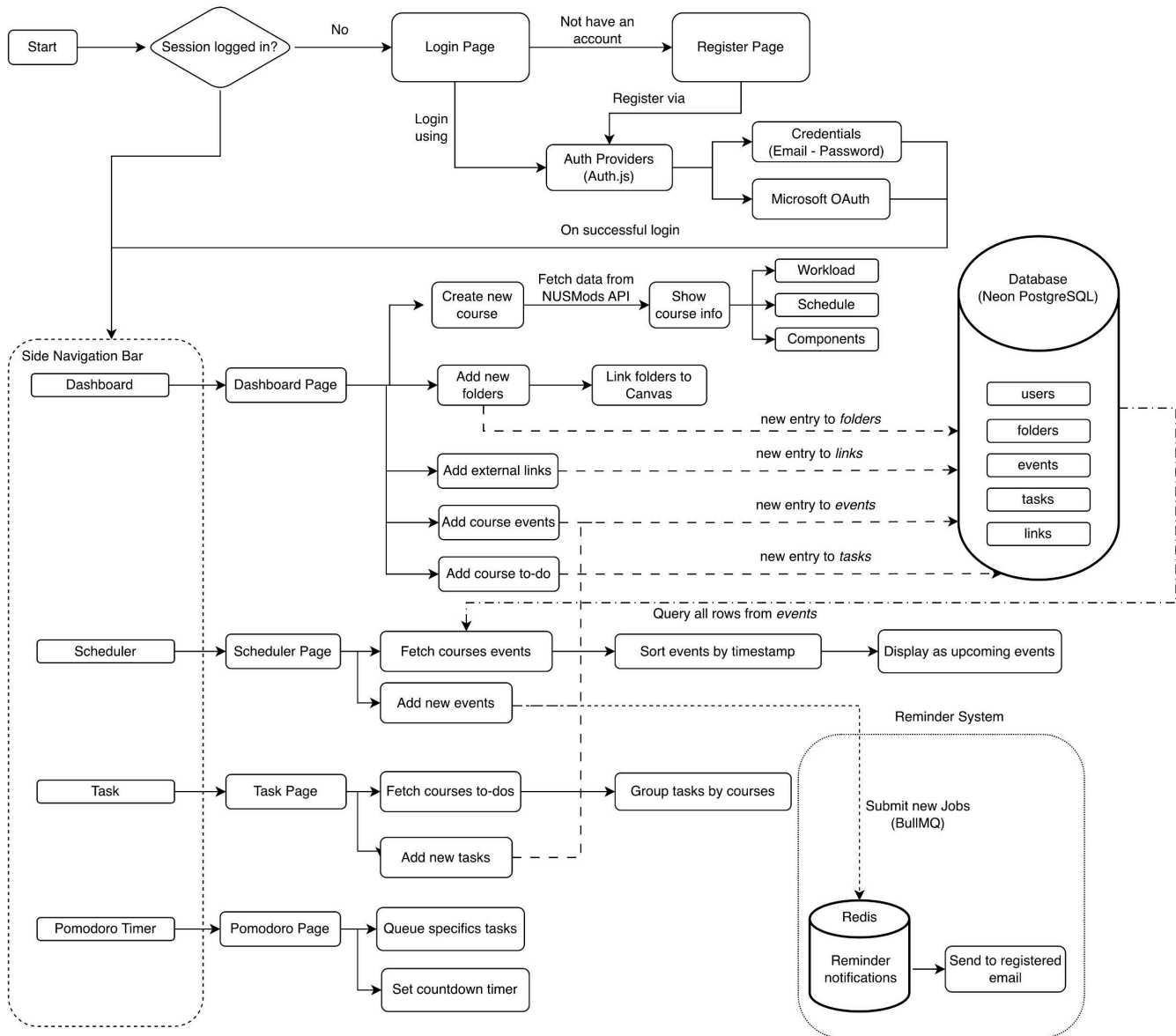


Figure 1. Overall System Architecture

The NUS Savnac system consists of four main components: the frontend application, backend services, database, and background job processing system. Users interact with the platform through a Next.js frontend, which communicates with backend services and external data sources such as the NUSMods API.

User data, tasks, events, folders, and course information are stored in a PostgreSQL database hosted on Neon. To support automated reminders, BullMQ and Redis are used to process background jobs and deliver scheduled email notifications. This architecture provides a clear separation between user interaction, data management, and asynchronous processing, ensuring maintainability and scalability.

2.2. Technology Stack

Frontend

- **Next.js** A React-based framework used to build the frontend application. Next.js was chosen for its modern routing system, server-side rendering capabilities, and seamless integration with the React ecosystem.

- **Tailwind CSS** A utility-first CSS framework used for styling the application. Tailwind CSS enables rapid UI development while maintaining a consistent and scalable design system.
- **shadcn/ui** A collection of reusable and customizable UI components built on top of Radix UI and Tailwind CSS. It was selected to accelerate development while maintaining full control over component styling and behavior.
- **Framer Motion** An animation library used to create smooth transitions and interactive user experiences. Framer Motion helps improve the overall usability and visual appeal of the application.

Backend

- **NestJS** A progressive Node.js framework used to build the backend services. NestJS was chosen for its modular architecture, strong TypeScript support, and maintainable project structure.
- **NUSMods API** An external API that provides module, timetable, and academic information from NUS. It serves as the primary source of module-related data within the application.

Database

- **PostgreSQL** A relational database management system used for persistent data storage. PostgreSQL was selected for its reliability, performance, and strong support for complex relational data.
- **Prisma** A type-safe ORM used to interact with the PostgreSQL database. Prisma simplifies database operations while improving developer productivity and reducing the likelihood of runtime errors.

Background Jobs

- **BullMQ** A Redis-based job queue used for scheduling and processing asynchronous tasks. BullMQ enables the application to handle background operations without affecting user-facing performance.
- **Redis** An in-memory data store used by BullMQ for queue management and job persistence. Redis provides high performance and low latency for background task processing.

Authentication

- **Auth.js** An authentication framework used to manage user authentication and session handling. Auth.js was chosen for its seamless integration with Next.js and support for multiple authentication providers.
 - *Credentials (Username and Password)* Provides traditional account-based authentication, allowing users to securely register and log in using their email and password.
 - *Microsoft OAuth* Enables users to authenticate using their Microsoft accounts, providing a convenient and secure single sign-on experience.

Deployment

- **Vercel** The hosting platform used for deploying the frontend application. Vercel offers optimized support for Next.js applications and provides a streamlined deployment workflow.
- **Railway** A cloud platform used to host backend services and supporting infrastructure. Railway simplifies service deployment and environment management.
- **Neon** A serverless PostgreSQL platform used to host the application's database. Neon was selected for its scalability, ease of management, and seamless integration with modern cloud workflows.

Developer Tools

- **Git/GitHub** Used for version control and collaborative development. Git and GitHub facilitate code management, feature branching, and team collaboration throughout the project lifecycle.
- **Jest** A testing framework used for writing and executing unit tests. Jest helps ensure the correctness and reliability of application logic.
- **Supertest** A library used for API and integration testing. Supertest allows backend endpoints to be tested in an automated and reproducible manner.

2.4. Authentication design

3. Feature Implementation

3.1. User Authentication

3.1.1. Feature Overview

3.1.2. Key Functionalities

3.1.3. Implementation Highlights